JAVA SDE-2 INTERVIEW PREPARATION SERIES

VOLUME 12 OF 18

# Binary Search: Bounds, Answers, and Invariants

## Exact Matches, Lower Bounds, Monotone Predicates, and Safe Midpoints


FOUNDING AUTHOR

**Vinay Reddy Kalluri**

EDITOR-IN-CHIEF | CHIEF AUDITOR


Stop memorizing one template: define the candidate interval, monotone predicate, progress rule, and boundary contract that make binary search correct.

BOUNDS | INVARIANTS | LOWER BOUND | ROTATION | ANSWER SEARCH

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION


Series edition - Release 2026-07-25

Open educational interview-preparation edition

# Start Here - Your Route Through This Volume

> **60-second entry rule:** If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to `11 – Stacks, Queues, and Deques`. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

Binary search is a proof pattern over monotone state. This volume centers interval semantics, duplicates, insertion points, overflow-safe midpoint calculation, and search on answers.

## Readiness map

| Decision | Evidence |
|---|---|
| **START HERE** | The search space is sorted or governed by a monotone predicate; A first, last, lower, or upper boundary is required; Trying an answer can be checked efficiently; Each comparison can eliminate a contiguous region. |
| **PREPARE FIRST** | Volumes 1-6; Complexity and loop invariants. |
| **MOVE ON WHEN** | Choose closed or half-open interval templates; Implement lower bounds and insertion points; Use Java binarySearch APIs without misreading negative results; Prove progress and handle duplicates safely. |

## Choose the route that fits today

1. **Learning:** read in order, type the representative Java examples, and complete each dry run.
2. **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

## Volume Position

| 11 - Stacks, Queues, and Deques | **YOU ARE HERE**<br>**12 - Binary Search** | 13 - Trees, BSTs, and Tries |
|---|---|---|

# Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION

Part I - Core Learning

SDE-2 CORE CHAPTER

# Chapter 1 - Binary Search: Bounds, Monotone Answers, and Invariants for SDE-2

Binary search is a proof pattern, not a remembered loop. The searchable object is an ordered decision boundary: all states on one side have one classification and all states on the other side have another. Sometimes the ordered domain is a sorted array. Sometimes it is a numeric capacity, time, rate, or precision interval whose feasibility predicate is monotone. At SDE-2 level, define the interval and invariant aloud, make every branch preserve them, and explain duplicates, overflow, and library return semantics.

## CHOOSE IT | Recognition map

| Prompt signal | Search target | Preferred template |
|---|---|---|
| find an exact value in sorted data | one matching index | classic exact search |
| first position not less than x | insertion boundary | lower bound |
| first position greater than x | insertion boundary | upper bound |
| first/last occurrence | duplicate range | lower/upper composition |
| smallest feasible capacity/time/rate | monotone false-to-true boundary | first true |
| largest feasible budget/threshold | monotone true-to-false boundary | last true or last false after inversion |
| array rotated from sorted order | one ordered half per iteration | rotated search |
| row/column order or flattened matrix order | ordered coordinates | matrix-specific binary search |
| local peak exists | slope points toward a peak | neighbor comparison |
| real-valued approximation | continuous interval | iteration/tolerance search |

Do not binary-search merely because the input is an array or because the answer is numeric. You need a total order or a monotone predicate. If feasibility changes `false, true, false`, discarding half the domain is unsound.

## Pick one interval convention

This chapter favors half-open intervals `[low, high)` for bounds. They represent an empty interval naturally when `low == high`, and `high` may be the array length as a valid insertion sentinel. For exact search it uses a closed interval `[low, high]`, because success terminates immediately and the empty condition `low > high` is familiar. Mixing update rules across these templates is a common source of nontermination.

### Overflow-safe midpoint

For nonnegative indexes where `low <= high`, compute:

```text
mid = low + (high - low) / 2
```

not `(low + high) / 2`, whose sum may overflow. For a `long` answer domain, even `high-low` can overflow if the interval spans nearly the entire signed range. Most interview domains are nonnegative and bounded by input sums; state that assumption or use a representation/unsigned technique suited to the full range.

## Family 1: exact search

### Invariant and proof

For a sorted ascending array, use a closed candidate interval. The invariant is:

> If the target exists in an unexamined position, at least one occurrence lies in `[low, high]`.

Compare `a[mid]` with target. Equality returns. If `a[mid] < target`, sortedness proves indexes `<= mid` cannot match, so assign `low = mid+1`. Otherwise assign `high = mid-1`. Each unsuccessful step removes `mid`, shrinking the interval. When `low > high`, no candidate remains.

Dry-run target `7` in `[1,3,5,7,9]`: `[0,4]` examines `5`, leaving `[3,4]`; it examines `7` and returns index `3`. Searching `6` follows `[0,4] -> [3,4] -> [3,2]`, then returns absent.

Time is `O(log n)`, auxiliary space `O(1)`. With duplicates, exact search may return any occurrence. If the contract requires first or last, use a boundary search rather than storing a match and improvising.

## Family 2: lower bound, upper bound, first, and last

### Lower bound

`lowerBound(a, x)` returns the first index whose value is at least `x`, or `a.length`. Maintain `[low, high)` with:

- all indexes before `low` have value `< x`;
- all indexes at or after `high` have value `>= x` (within the array);
- the boundary lies in `[low, high]`.

If `a[mid] < x`, move `low` to `mid+1`; otherwise move `high` to `mid`. At equality of boundaries, `low` is the first legal insertion point.

### Upper bound

`upperBound(a, x)` returns the first index whose value is greater than `x`. The only comparison change is that values `<= x` move `low` right. Therefore occurrences occupy half-open range `[lowerBound, upperBound)`, count is `upper−lower`, first is lower when in range, and last is `upper−1` when a match exists.

Dry-run `[1,2,2,2,4]`, target `2`: lower bound converges to `1`; upper bound converges to `4`. The first/last range is indexes `1..3`, and insertion before equals uses `1` while insertion after equals uses `4`.

Bounds handle empty arrays and edge insertion naturally. They also compose with range-counting and sorted deduplication. Do not access `a[bound]` before checking `bound < a.length`.

## Family 3: first true and last false

Many "binary search on answer" problems reduce to a monotone predicate over integer candidates:

```
TEXT
false false false true true true
                 ^ first true
```

Use `[low, high)` and a virtual true sentinel at `high` if no real candidate succeeds. The invariant is that candidates below `low` are known false and candidates at/above `high` are known true or sentinel. If `predicate(mid)` is true, retain it by setting `high=mid`; otherwise discard it with `low=mid+1`. The return may equal the original high-exclusive value, meaning no true candidate.

`lastFalse` is `firstTrue − 1` when the false region may be empty. That sentinel is not representable when `firstTrue` is `Integer.MIN_VALUE`; the sample uses exact subtraction and rejects that case instead of wrapping. A production API can return an optional boundary when an empty false region is valid. If the requested pattern is true-to-false, invert the predicate or derive an explicit last-true template and prove its progress. Avoid memorizing mirrored updates without a boundary model.

The predicate's cost matters: total time is `O(P * log R)`, where `P` is one predicate evaluation and `R` is numeric range width. Predicate calls must not mutate shared state in a way that changes later truth values.

## Family 4: rotated arrays, including duplicates

For a strictly increasing array rotated once, at least one half around `mid` is sorted. If `a[low] <= a[mid]`, the left half is sorted. Check whether target lies within its ordered bounds; keep that half or discard it. Otherwise the right half is sorted and the symmetric test applies.

Duplicates can hide which half is informative: with `a[low] == a[mid] == a[high]`, either side may contain the pivot or target. Shrink both endpoints. This preserves correctness but can degrade worst-case time to `O(n)`, as in an array of mostly equal values. Be explicit: binary search does not guarantee logarithmic time when duplicates erase ordering information.

Dry-run `[2,5,6,0,0,1,2]`, target `0`: middle index `3` matches immediately. For target `3`, branches eliminate ordered ranges and eventually return false. On `[1,0,1,1,1]`, endpoint shrinking may be required before the sorted half becomes visible.

## Family 5: peak finding

A peak is an element not smaller than its neighbors under the problem's exact definition. For the common strict adjacent-inequality version, compare `a[mid]` with `a[mid+1]` while searching `[low, high]`. If the slope descends (`a[mid] > a[mid+1]`), a peak exists at `mid` or left, so set `high=mid`. If it ascends, a peak exists to the right, so set `low=mid+1`.

Why can a half be discarded? Following an ascending slope right cannot fall off the array without encountering a peak; following a descending slope left has the symmetric guarantee. The candidate interval always contains a peak and shrinks until one index remains. Time `O(log n)`, space `O(1)`. Duplicate plateaus require a changed peak definition or may destroy the strict directional argument.

## Family 6: matrix search

If every row is sorted and the first value of each row is greater than the previous row's last value, the matrix is globally sorted in row-major order. Treat virtual index `i` as `(i / columns, i % columns)` and run exact binary search. Validate rectangular shape. Compute total cells with checked multiplication; coordinate arithmetic must not overflow.

For `[[1,3,5],[7,9,11]]`, virtual indexes `0..5` map to the sorted sequence. Searching `9` examines virtual `2` (`5`), then `4` (`9`). Complexity is `O(log(rows*cols))`, space `O(1)`.

If rows and columns are sorted independently but row ranges overlap, flattening is invalid. Use the top-right staircase: too large moves left, too small moves down, giving `O(rows+cols)`. State which matrix contract you have.

## Family 7: monotone answer search

Suppose packages in fixed order must be shipped within `days`. For capacity `C`, simulate days required. If `C` is feasible, every larger capacity is feasible; if it is infeasible, every smaller one is infeasible. This is the monotonicity proof.

The smallest capacity lies between the maximum single weight and the sum of weights. Use `long`: a sum can overflow `int`. Predicate invariant: current load never exceeds `C`; when the next package would exceed it, start a new day, preserving order. Stop early when days used exceeds the limit.

Dry-run weights `[3,2,2,4,1,4]`, days `3`. Lower bound is `4`, upper `16`. Capacity `10` needs two days, so search left. Capacity `7` needs three and is feasible. Capacity `5` needs four and fails. Capacity `6` needs three, so the minimum is `6`.

Total cost is `O(n log(sum−max+1))`, auxiliary space `O(1)`. Answer search is often the right SDE-2 move when constructing the optimum directly is hard but checking a proposed answer is simple.

### Precision search over real values

Real-valued binary search cannot rely on `low+1`. Stop after a fixed iteration count, on an absolute/relative interval tolerance, or both. Fixed iterations provide a deterministic error contraction; after `k` steps, interval width is initial width divided by `2^k`. For IEEE-754 `double`, around 100 iterations is generally beyond useful precision for ordinary ranges.

For square root of nonnegative `x`, predicate `mid*mid >= x` risks overflow. Compare `mid >= x/mid` for positive mid, or control the interval and special cases. Define behavior for NaN, infinities, negative input, signed zero, and acceptable error. If exact integer root is requested, use integer arithmetic and division-based comparisons instead.

### Java library semantics

`Arrays.binarySearch(array, key)` returns some matching index if found. If absent, it returns `−(insertionPoint) − 1`, where insertion point is the first position at which the key could be inserted while preserving order. Recover it with `int insertion = −result − 1` only when `result < 0`.

With duplicates, the returned matching index is not promised to be first or last. The array/range must already be sorted under the same ordering. Object overloads and `Collections.binarySearch` depend on comparator consistency; sorting with one comparator and searching with another invalidates the precondition.

## Complete Java 21 reference implementation

Compile and run with `java -ea BinarySearchSde2`.

JAVA (continued 1/6)

```java
import java.util.Arrays;
import java.util.OptionalInt;
import java.util.function.IntPredicate;

public final class BinarySearchSde2 {
    private BinarySearchSde2() {}

    public static int exactSearch(int[] sorted, int target) {
        requireArray(sorted);
        int low = 0, high = sorted.length - 1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (sorted[mid] == target) return mid;
            if (sorted[mid] < target) low = mid + 1;
            else high = mid - 1;
        }
        return -1;
    }

    public static int lowerBound(int[] sorted, int target) {
        requireArray(sorted);
        int low = 0, high = sorted.length;
        while (low < high) {
            int mid = low + (high - low) / 2;
            if (sorted[mid] < target) low = mid + 1;
            else high = mid;
        }
        return low;
    }

    public static int upperBound(int[] sorted, int target) {
        requireArray(sorted);
        int low = 0, high = sorted.length;
        while (low < high) {
            int mid = low + (high - low) / 2;
            if (sorted[mid] <= target) low = mid + 1;
```

JAVA (continued 2/6)

```
            else high = mid;
        }
        return low;
    }

    public static OptionalInt firstIndex(int[] sorted, int target) {
        int index = lowerBound(sorted, target);
        return index < sorted.length && sorted[index] == target
                ? OptionalInt.of(index) : OptionalInt.empty();
    }

    public static OptionalInt lastIndex(int[] sorted, int target) {
        requireArray(sorted);
        int index = upperBound(sorted, target) - 1;
        return index >= 0 && sorted[index] == target
                ? OptionalInt.of(index) : OptionalInt.empty();
    }

    public static int firstTrue(int lowInclusive, int highExclusive,
                                IntPredicate monotonePredicate) {
        if (lowInclusive > highExclusive || monotonePredicate == null) {
            throw new IllegalArgumentException("invalid range or predicate");
        }
        int low = lowInclusive, high = highExclusive;
        while (low < high) {
            int mid = (int) (low + ((long) high - low) / 2);
            if (monotonePredicate.test(mid)) high = mid;
            else low = mid + 1;
        }
        return low;
    }

    public static int lastFalse(int lowInclusive, int highExclusive,
                                IntPredicate monotonePredicate) {
        return Math.subtractExact(
```

JAVA (continued 3/6)

```
                firstTrue(lowInclusive, highExclusive, monotonePredicate), 1);
    }

    public static boolean searchRotatedWithDuplicates(int[] values, int target) {
        requireArray(values);
        int low = 0, high = values.length - 1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            if (values[mid] == target) return true;
            if (values[low] == values[mid] && values[mid] == values[high]) {
                low++;
                high--;
            } else if (values[low] <= values[mid]) {
                if (values[low] <= target && target < values[mid]) high = mid - 1;
                else low = mid + 1;
            } else {
                if (values[mid] < target && target <= values[high]) low = mid + 1;
                else high = mid - 1;
            }
        }
        return false;
    }

    public static int peakIndex(int[] values) {
        requireArray(values);
        if (values.length == 0) throw new IllegalArgumentException("empty array");
        int low = 0, high = values.length - 1;
        while (low < high) {
            int mid = low + (high - low) / 2;
            if (values[mid] > values[mid + 1]) high = mid;
            else low = mid + 1;
        }
        return low;
    }
```

JAVA (continued 4/6)

```
    public static boolean searchRowMajorMatrix(int[][] matrix, int target) {
        int columns = validateRectangular(matrix);
        if (matrix.length == 0 || columns == 0) return false;
        int cells = Math.multiplyExact(matrix.length, columns);
        int low = 0, high = cells - 1;
        while (low <= high) {
            int mid = low + (high - low) / 2;
            int value = matrix[mid / columns][mid % columns];
            if (value == target) return true;
            if (value < target) low = mid + 1;
            else high = mid - 1;
        }
        return false;
    }

    public static long minimumShipCapacity(int[] weights, int days) {
        requireArray(weights);
        if (weights.length == 0 || days <= 0) {
            throw new IllegalArgumentException("nonempty weights and positive days required");
        }
        long low = 0, high = 0;
        for (int weight : weights) {
            if (weight <= 0) throw new IllegalArgumentException("weights must be positive");
            low = Math.max(low, weight);
            high = Math.addExact(high, weight);
        }
        while (low < high) {
            long mid = low + (high - low) / 2;
            if (canShip(weights, days, mid)) high = mid;
            else low = mid + 1;
        }
        return low;
    }

    private static boolean canShip(int[] weights, int allowedDays, long capacity) {
```

JAVA (continued 5/6)

```
        int usedDays = 1;
        long load = 0;
        for (int weight : weights) {
            if (load + weight > capacity) {
                usedDays++;
                load = 0;
                if (usedDays > allowedDays) return false;
            }
            load += weight;
        }
        return true;
    }

    public static double squareRoot(double value, double relativeTolerance) {
        if (!Double.isFinite(value) || value < 0
                || !(relativeTolerance > 0) || !Double.isFinite(relativeTolerance)) {
            throw new IllegalArgumentException("finite nonnegative value/tolerance required");
        }
        if (value == 0 || value == 1) return value;
        double low = 0, high = Math.max(1.0, value);
        for (int iteration = 0; iteration < 200; iteration++) {
            double mid = low + (high - low) / 2.0;
            if (mid >= value / mid) high = mid;
            else low = mid;
            if (high - low <= relativeTolerance * Math.max(1.0, high)) break;
        }
        return (low + high) / 2.0;
    }

    private static void requireArray(int[] values) {
        if (values == null) throw new IllegalArgumentException("array is null");
    }

    private static int validateRectangular(int[][] matrix) {
        if (matrix == null) throw new IllegalArgumentException("matrix is null");
```

JAVA (continued 6/6)

```
        if (matrix.length == 0) return 0;
        if (matrix[0] == null) throw new IllegalArgumentException("null row");
        int columns = matrix[0].length;
        for (int[] row : matrix) {
            if (row == null || row.length != columns) {
                throw new IllegalArgumentException("matrix must be rectangular");
            }
        }
        return columns;
    }

    public static void main(String[] args) {
        int[] data = {1, 2, 2, 2, 4, 7, 9};
        assert exactSearch(data, 7) == 5 && exactSearch(data, 6) == -1;
        assert lowerBound(data, 2) == 1 && upperBound(data, 2) == 4;
        assert firstIndex(data, 2).orElseThrow() == 1;
        assert lastIndex(data, 2).orElseThrow() == 3;
        assert firstTrue(0, 10, x -> x * x >= 30) == 6;
        assert lastFalse(0, 10, x -> x >= 6) == 5;
        assert firstTrue(Integer.MIN_VALUE, Integer.MAX_VALUE, x -> x >= 0) == 0;

        assert searchRotatedWithDuplicates(new int[] {2, 5, 6, 0, 0, 1, 2}, 0);
        assert !searchRotatedWithDuplicates(new int[] {2, 5, 6, 0, 0, 1, 2}, 3);
        int[] mountain = {1, 3, 5, 4, 2};
        assert peakIndex(mountain) == 2;

        int[][] matrix = {{1, 3, 5}, {7, 9, 11}};
        assert searchRowMajorMatrix(matrix, 9) && !searchRowMajorMatrix(matrix, 8);
        assert minimumShipCapacity(new int[] {3, 2, 2, 4, 1, 4}, 3) == 6;
        assert Math.abs(squareRoot(2, 1e-12) - Math.sqrt(2)) < 1e-10;

        int library = Arrays.binarySearch(data, 6);
        assert library < 0 && -library - 1 == 5;
    }
}
```

## Complexity and contract matrix

| Operation | Time | Space | Contract caveat |
|---|---|---|---|
| exact/lower/upper bound | `O(log n)` | `O(1)` | input sorted under same order |
| first true | `O(log R)` predicate calls | `O(1)` | predicate monotone on domain |
| rotated with duplicates | average `O(log n)`, worst `O(n)` | `O(1)` | one rotation of nondecreasing data |
| peak | `O(log n)` | `O(1)` | adjacent/peak definition supports slope proof |
| row-major matrix | `O(log(rc))` | `O(1)` | rectangular and globally row-major sorted |
| minimum capacity | `O(n log S)` | `O(1)` | positive weights, fixed ordering |
| real square root | `O(log(range/error))` or capped | `O(1)` | floating-point error contract |

`R` is discrete range size and `S` is the capacity search interval. State predicate cost rather than calling every answer search simply `O(log n)`.

## WATCH OUT | Edge cases and common mistakes

- Empty arrays: exact search is absent; lower and upper bound are zero.
- Duplicates: arbitrary match, first, and last are different contracts.
- Half-open templates use `high=mid`; closed templates often use `high=mid−1`. Do not mix them.
- A loop that assigns `low=mid` can stall when two candidates remain. Bias the midpoint or use first-true form.
- Check a returned insertion index before array access.
- Sorting order and comparator used for search must agree.
- Rotated duplicates may force linear work; do not claim unconditional logarithmic time.
- A matrix sorted only by rows cannot be flattened into one sorted sequence.
- Sum-based upper bounds and products need `long` or exact arithmetic.
- Feasibility must be monotone and preferably side-effect-free.
- Floating-point search needs tolerance/iteration and special-value policy, not equality with a computed real.

## TRY IT | Exercises with model checkpoints

### Exercise 1: count values in a range

Count sorted-array values in inclusive numeric range `[a,b]`.

**Checkpoint:** if `a>b`, return zero. Answer is `lowerBound(first > b)` minus `lowerBound(first >= a)`, i.e. `upperBound(b)−lowerBound(a)`. The half-open index interval makes duplicates automatic.

### Exercise 2: search a rotated array for first physical occurrence

The array contains duplicates and the contract asks for the smallest index containing target.

**Checkpoint:** ordinary rotated search can prove existence but not first physical index without more work. Discuss worst-case `O(n)` and whether finding the pivot plus bound searches is valid under duplicates. Optimize only after making the index contract explicit.

### Exercise 3: minimum eating rate

Find the smallest integer rate that finishes piles within `h` hours.

**Checkpoint:** rate domain is `[1,maxPile]`; hours at rate `r` is sum of ceilings `(pile+r−1)/r`, computed overflow-safely as `(pile−1)/r+1` for positive piles. Hours is nonincreasing, so feasibility `hours<=h` is false-to-true as rate grows. Stop the sum early above `h`.

### Exercise 4: integer square root

Return floor square root for a nonnegative `long`.

**Checkpoint:** search for last `m` with `m <= x/m`, handling zero separately. Do not test `m*m<=x`, which can overflow. Use a closed or boundary template with a documented upper limit.

### Exercise 5: matrix contract comparison

Implement both row-major search and top-right staircase search.

**Checkpoint:** provide a matrix sorted by every row and column that violates global row-major order. Show that flattening can discard the target incorrectly, while staircase search uses column/row monotonicity correctly.

### Exercise 6: test a monotone predicate

Build a property test for an answer-search predicate.

**Checkpoint:** for sampled `x<y`, assert that `predicate(x)` implies `predicate(y)` for false-to-true monotonicity. This cannot prove all inputs, but catches stateful predicates and arithmetic errors. Separately compare binary results to a linear oracle on small randomized domains.

## SDE-2 production follow-ups

**How do you search remote or expensive data?** The number of comparisons is logarithmic, but each comparison may be a network round trip or disk read. Batch/prefetch nearby metadata, cache immutable results, enforce deadlines, and reconsider whether an index service should expose a direct bound operation.

**Can the collection change during search?** Mutation can invalidate sortedness and indexes. Use immutable snapshots, version checks, locks, or a data structure with defined concurrent navigation. Visibility alone does not preserve a multi-step search invariant.

**How do you prove an answer predicate?** Separate necessity and sufficiency. Show smaller candidates cannot work when current fails, and larger candidates cannot fail when current works. Then audit overflow and state mutation, because either can make observed truth nonmonotone even when the mathematics is monotone.

**How do you test boundaries?** Compare against a linear oracle on small arrays; generate empty, singleton, all-equal, minimum/maximum integer, target-below, target-above, duplicate-run, rotated, and no-solution cases. Assert both returned result and postconditions such as insertion partition.

## WHY IT WORKS | Interview proof clinic and model answers

**Why does lower bound set `high=mid` rather than `mid−1`?** When `a[mid] >= target`, `mid` itself may be the first qualifying index and must remain a candidate. The half-open interval permits retaining it by moving the exclusive upper boundary to `mid`. When `a[mid] < target`, `mid` cannot qualify, so `low=mid+1` discards it. These updates preserve the partition invariant and always shrink the interval.

**How do you know first-true returns the minimum feasible answer?** At termination `low==high`. The invariant says everything below low is proven false, and high is proven true or is the no-answer sentinel. Therefore no smaller feasible candidate exists, and if the result is not the sentinel it is feasible. This proves minimality, not only that some feasible value was found.

**Can a stateful predicate break search?** Yes. If caching, rate limits, random choices, overflow, or mutation changes its answer between calls, the observed sequence may not be monotone. Isolate state, use immutable inputs, make arithmetic wide/exact enough, and test predicate monotonicity separately from the search loop.

**When is interpolation or exponential search relevant?** Exponential search finds an upper boundary when the sorted domain is conceptually unbounded: probe `1,2,4,...` until crossing, guarding overflow, then binary-search the bracket. Interpolation search estimates a position from values but depends on distribution and can degrade badly; ordinary binary search is the dependable baseline.

**How would you search a versioned remote API?** Pin all predicate/element reads to one immutable version. Otherwise concurrent updates can change the ordering between comparisons. Count network round trips, apply deadlines and retries only when safe, and prefer a server-side lower-bound endpoint because asymptotically few calls can still be operationally expensive.

**What is the right answer when no candidate is feasible?** Encode it explicitly: optional result, sentinel outside the legal domain, or exception for violated preconditions. The first-true helper returns `highExclusive` as a sentinel, but a domain API should translate that into a type the caller cannot mistake for a real capacity. Test all-false and all-true domains.

### A complete boundary-review checkpoint

Before submitting, write three examples beside the loop: empty domain, one candidate, and two candidates. Trace whether the midpoint and update remove or retain the examined candidate. Then assert the postcondition, not merely one expected index: every value before lower bound is smaller and every value at/after it is not smaller; every capacity below the returned minimum is infeasible and the returned capacity is feasible. These partition assertions expose bugs that favorable examples miss. For a costly predicate, cache only if candidates and input version fully determine the result; otherwise stale state can violate the proof. Finally, translate the generic sentinel into a domain result before returning from a public API.

## Final readiness checklist

- I state the domain, ordering, interval convention, and invariant first.
- Every branch discards `mid` or safely retains a boundary while guaranteeing progress.
- I distinguish exact match, lower bound, upper bound, first, and last.
- I prove predicate monotonicity before answer search.
- I include predicate cost and range width in complexity.
- I audit duplicates, sentinels, arithmetic overflow, and library encoding.
- For matrices and peaks, I use the actual structural guarantee-not an imagined global sort.

Binary search becomes reliable when every assignment is justified by a boundary proof. Once that habit is learned, capacities, deadlines, rates, and precision problems become the same disciplined search in different clothing.

Part II - Practice and Handoff

# Practice Ladder and Next Steps

## TRY IT | Practice ladder

- Foundation: exact match and insertion point
- Interview Core: first/last occurrence and rotated arrays
- SDE-2 Follow-up: binary search on answer
- Challenge: design and prove a monotone feasibility predicate

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

### Navigation

Previous volume: 11 - Stacks, Queues, Deques, and Monotonic Patterns

Series index

Next volume: 13 - Trees, Binary Search Trees, and Tries

## TRY IT | Completion check

- [ ] I can recognize the core patterns without being told the data structure or algorithm.
- [ ] I can implement the representative Java templates from memory.
- [ ] I can explain why each implementation is correct.
- [ ] I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- [ ] I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

# Series Roadmap

Complete the learning steps in order when rebuilding from fundamentals, or enter at the first step whose readiness checks expose a gap. Stable volume numbers remain in filenames; the steps below show the recommended reading order. Click a title when your PDF viewer permits local sibling-file links.

| STEP | FOCUSED LEARNING PATH |
|---|---|
| 1 | Java Foundations for Problem Solving |
| 2 | Time and Space Complexity |
| 3 | Number Systems and Math Foundations for DSA Interviews |
| 4 | Bit Manipulation in Java |
| 5 | Loop Mastery, Patterns, and Index Calculations |
| 6 | Arrays and Array Problem-Solving Patterns |
| 7 | Strings and String Problem-Solving Patterns |
| 8 | Hashing: Maps, Sets, Frequency, and Prefix State |
| 9 | Recursion and Backtracking in Java |
| 10 | Linked Lists: Pointer Reasoning and Mutation |
| 11 | Stacks, Queues, Deques, and Monotonic Patterns |
| 12 | Binary Search: Bounds, Answers, and Invariants |
| 13 | Trees, Binary Search Trees, and Tries |
| 14 | Heaps, Priority Queues, Selection, and Top-K |
| 15 | Graphs: Traversal, Ordering, Paths, and Union-Find |
| 16 | Greedy Algorithms: Recognition, Proofs, and Scheduling |
| 17 | Dynamic Programming: State, Transitions, and Optimization |
| 18 | Advanced Java Engineering and the SDE-2 Interview (Parts A-J) |

Learning Step 3 uses a linked Number Systems foundations volume and workbook; the final advanced stage uses a ten-part collection, including a three-volume backend specialist track. These splits keep every file focused and printable.

# About the Author

## Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer, the creator and founding author of the Java SDE-2 Interview Preparation Series, and its Editor-in-Chief and Chief Auditor. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

## Editorial leadership

As Editor-in-Chief, Vinay owns the learning sequence, scope, voice, and publication decisions. As Chief Auditor, he owns the Java accuracy standard, validation evidence, attribution review, and release-readiness gate. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

## Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

## Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

**Connect:** LinkedIn | GitHub

# Copyright and Publishing Notes



Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Volume 12 of 18. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.